

Duolingo — cercetare pentru arhitectura Platformei BAC

Document de lucru, scris la 2026-08-07. Sursă de inspirație, **nu** de copiat. Fiecare secțiune se termină cu „**Ce înseamnă pentru noi**” — traducerea concretă în platforma noastră, cu ce avem deja în DB și ce ne lipsește. Surse la final; cifrele sunt date, pentru că îmbătrânesc repede.

0. De ce Duolingo e un model bun — și unde nu e

Duolingo e cea mai bine documentată aplicație de învățare din lume: publică lucrări științifice, blog de inginerie, iar din 2021 e companie listată, deci raportează cifre auditate. Poți învăța din ea fără să ghicești.

Scara (FY2025): peste **50 milioane DAU**, 133,1 milioane MAU, **12,2 milioane abonați plători**, peste **1 miliard \$ bookings**, peste 400 milioane \$ profit net. Rata de conversie free → plătit: **~9,2%** din MAU.

Dar atenție la diferența fundamentală de produs:

	Duolingo	Platforma BAC
Orizont	nelimitat — înveți „pentru totdeauna”	fix: data examenului
Motivație	intrinsecă, fragilă — trebuie fabricată	extrinsecă, deja puternică (examenul)
Curriculum	proprietar, îl decid ei	impus de programa MEN
Succes =	ai revenit azi (DAU)	ai luat examenul
Abandon	userul pleacă, pierzi un abonat	elevul pică examenul

Asta e cea mai importantă concluzie a întregii cercetări: **Duolingo își petrece 80% din efortul de produs fabricând motivația pe care voi o primiți gratis.** Un elev de clasa a XII-a în martie nu are nevoie să fie convins să învețe. Are nevoie să știe **ce** să învețe și **unde stă prost**. Copiate orbește, mecanicile de retenție ale Duolingo ar fi zgomot — în cel mai rău caz, infantilizante.

Ce merită copiat e altceva: **modelul de date, bucla de repetiție și disciplina de măsurare.**

1. Modelul de conținut — ierarhia

Duolingo a trecut în 2022 de la „skill tree” (arbore ramificat, alegi singur pe unde mergi) la „**path**” — un **drum liniar, obligatoriu**. E cea mai relevantă decizie de produs pentru noi.

Curs

- └ Section ~ aliniată la un nivel CEFR (A1, A2, B1, B2)
 - └ Unit ~10 niveluri, o temă (vocabulary „la restaurant” / gramatică)
 - └ Level ~ un pas pe hartă, are un tip (vezi mai jos)
 - └ Lesson ~ până la 17 exerciții

Tipurile de nivel sunt marcate vizual prin iconițe: stea (lecție standard), ganteră (practică personalizată), trofeu (recapitulare de unitate), carte (poveste), căști (audio), bule de dialog (roleplay).

De ce au renunțat la arbore: în arbore, elevul alegea singur ordinea și sărea peste ce nu-i plăcea; doar lecțiile de dificultate mică erau obligatorii. Pe drum liniar, lecțiile obligatorii acoperă trei niveluri de dificultate. Rezultatul măsurat: drumul liniar **duce la rezultate mai bune de proficiență** decât arborele.

Lecția: libertatea de navigare pare prietenoasă, dar produce goluri. Elevii evită exact ce nu știu.

Ce înseamnă pentru noi

Ierarhia noastră actuală e `chapters → lessons`, două niveluri. Duolingo are patru. Nu ne trebuie patru, dar **ne lipsește un nivel intermediar** dacă un capitol de BAC (ex. „Funcții”) are 30 de lecții — devine o listă lungă și demoralizantă.

Mai important: noi avem `order_index` pe lecții, dar **nimic nu forțează ordinea**. Elevul poate deschide orice lecție. Întrebarea de discutat: vrem drum liniar cu deblocare progresivă, sau acces liber? Duolingo a măsurat că liniarul e mai bun pedagogic — dar la ei conținutul e secvențial prin natura lui (nu poți face conjunctivul înainte de prezent). La BAC, un elev care vrea doar „Integrale” înainte de teză are un motiv legitim. **Recomandarea mea: drum recomandat implicit, dar nu blocat** — cu marcaj clar „ai sărit peste 3 lecții din acest capitol”.

2. Bucla de învățare și repetiția

Half-Life Regression (HLR) — 2016

Lucrarea `A Trainable Spaced Repetition Model for Language Learning` (Settles & Meeder, ACL 2016) e piesa centrală și e **publică, cu cod și date** (13 milioane de trasee de învățare, pe GitHub și Harvard Dataverse).

Ideea: pentru fiecare element învățat se estimează un **„timp de înjumătățire” în memoria de lungă durată** — după cât timp probabilitatea de reamintire scade la 50%. Modelul combină teoria psiholingvistică a curbei uitării cu regresie antrenată pe date reale, în loc de intervale fixe (cum are Anki/SM-2).

Rezultate: eroare de predicție cu **45%+ mai mică** decât metodele de referință, și **+12% engagement zilnic** într-un test A/B real.

Birdbrain — 2020, acum v2

Motorul de personalizare. După **fiecare** exercițiu, estimează simultan două lucruri: **cât de greu e exercițiul** și **cât de bun e elevul**. Matematica vine din *Item Response Theory* (psihometrie) — regresie logistică în care probabilitatea de răspuns corect e funcție de (dificultate, abilitate).

Scopul declarat: „**Goldilocks difficulty**” — prea ușor plictisește, prea greu alungă. Sistemul țintește zona în care elevul reușește, dar cu efort.

Practice Hub

Un spațiu separat de recapitulare, alimentat exclusiv din **activitatea recentă a elevului**: o secțiune „Greșeli” care readuce exact ce a greșit, plus o sesiune ținută care se reîmprospătează zilnic.

Ce înseamnă pentru noi

Aici e **cea mai valoroasă idee din toată cercetarea**, și avem o problemă concretă de arhitectură care ne blochează:

`student_progress` face **upsert** și **suprascrie**. Reținem `score`, `total`, `attempts` per (elev, capitol) — dar **nu reținem ce a răspuns elevul la fiecare întrebare, nici când**. Fără istoric la nivel de întrebare, nu putem construi nici repetiție spațiată, nici „exersează ce ai greșit”, nici estimare de dificultate pe întrebare.

Ne-ar trebui un tabel de tip `answer_events` (elev, întrebare, răspuns ales, corect, timestamp) — append-only, o linie per răspuns. Din el ies gratis:

- **„Greșelile mele”** — cea mai utilă funcție posibilă pentru un elev de BAC, și cea mai ieftină de construit;
- **dificultatea reală per întrebare** (% de elevi care greșesc) — care îi spune profesorului ce să reexplice, fără niciun ML;
- **baza pentru repetiție spațiată**, dacă vrem mai târziu.

Un IRT complet e exagerat pentru noi. Dar **„procentul de elevi care au greșit întrebarea X”** e un `GROUP BY` și valorează enorm.

3. Gamificare — ce funcționează și de ce

Toate mecanicile lor au o componentă de **aversiune la pierdere**: durerea de a pierde ceva construit e mai puternică decât plăcerea de a câștiga ceva nou.

Mecanică	Cum funcționează	Efect măsurat
Streak	zile consecutive; se pierde la o zi ratată	7 zile de streak → 3,6× mai probabil să rămână pe termen lung
Streak Freeze	protejează streak-ul o zi	-21% churn pentru userii cu streak în pericol
Streak Wager	pariu pe menținerea streak-ului	+14% retenție la ziua 14
XP	monedă unică pentru orice activitate	leagă toate sistemele între ele
Ligi	clasament săptămânal pe XP, promovare/retrogradare	transformă învățatul în competiție cu necunoscuți
Hearts	vieți limitate; greșelile costă	monetizare (Super = vieți nelimitate)
Badges	realizări	+2,4% DAU , +4,5% sesiuni terminate, +116% prieteni adăugați

Coerența e cheia: **XP e moneda comună** — o lecție terminată avansează simultan streak-ul, poziția în ligă și progresul spre insigne. De asta sistemul pare un întreg, nu o colecție de funcții lipite.

Ce înseamnă pentru noi

Ce aș lua:

- **Streak-ul**, dar redefinit: nu „zile consecutive pe aplicație”, ci **zile în care ai învățat ceva**. La 3 luni de examen, consecvența e chiar factorul care decide. Și **obligatoriu cu ceva de tip „streak freeze”** — un elev care învață 6 zile și ratează duminica nu trebuie pedepsit, altfel abandonează complet („oricum am pierdut totul”).
- **Progres vizibil per capitol** — bara „știi 60% din Funcții”. Elevul de BAC nu întreabă „câte XP am”, ci „sunt pregătit?”.

Ce NU aș lua:

- **Hearts / vieți**. La Duolingo e mecanism de monetizare: greșești → pierzi vieți → plătești. La o platformă de examen e **contraproductiv și cinic**: pedepsești exact comportamentul pe care vrei să-l încurajezi (să exersezi mult, să greșești în siguranță acasă, nu la examen).
- **Ligi cu necunoscuți**. Un elev de a XII-a stresat n-are nevoie să fie retrogradat public. Eventual, mult mai târziu, clasament **în cadrul clasei**, opțional.
- **XP ca scop în sine**. La ei XP e proxy pentru „ai revenit”. La voi există o metrică reală și onestă: **scorul la teste pe capitol**.

Regula pe care aș aplica-o: **fiecare element de gamificare trebuie să răspundă la „mă apropie de notă mai mare la BAC?”**. Dacă răspunsul e „nu, dar crește engagement-ul”, e zgomot — la voi engagement-ul nu e problema.

4. Notificări

Duolingo tratează notificarea ca produs, nu ca detaliu. Elemente concrete:

- **Punctul roșu** pe iconiță: **+6% DAU** din șase rânduri de cod. A doua versiune, încă +1,6%.
- **Momentul**: reamintirea vine la **23,5 ore** de la ultima sesiune — deci fereastra zilnică se „deplasează” ușor înainte, nu ratează ora.
- **Tonul**: mesaj personalizat („Hi, it's Duo”) — doar optimizarea textului a adus **+5% DAU**.
- **Infrastructura**: sute de microservicii pe AWS ECS; pentru campania de Super Bowl au construit un sistem care trimite **4 milioane de notificări în 5 secunde** (~800k/s), cu cozi SQS FIFO pentru deduplicare (idempotență — aceeași lecție ca la webhook-ul nostru Stripe).

Ce înseamnă pentru noi

Nu ne trebuie 800k/s. Ne trebuie **una singură, la momentul potrivit**: emailul către elev când profesorul i-a răspuns la tichet — exact ce e blocat acum în TASKS.md.

Ce merită preluat e **principiul**: notificarea nu e „anunț”, e **declanșator de revenire**. Iar la un produs de examen, notificarea cu adevărat valoroasă nu e „ai un streak de apărat”, ci **„mai sunt 47 de zile până la BAC și n-ai atins deloc capitolul Integrale”**.

5. Arhitectură tehnică

Session Generator — rescrierea în Scala

Piesa centrală a backendului lor: modulul care decide **ce exerciții vede userul și în ce ordine**, pentru 88+ cursuri. Exista din prima zi și acumulasе datorie tehnică.

Problemele vechii versiuni (Python):

- multe **dependențe pe resurse partajate** → apeluri de rețea în plus → latență;
- Python: mai lent, limitări de thread-safety (deci fără cache în memorie eficient), tipare dinamică → bug-uri descoperite la runtime, ciclu „deploy → găsești bug → repari”.

Principiul rescrierii, unul singur: „*elimină cât mai multe resurse partajate*”. Datele de curs sunt **procesate offline și serializate în fișiere pe S3**, deci se pot ține în memorie; doar datele de personalizare ale userului vin prin API.

Rezultat: latență de la 750ms la 14ms (–98%), iar disponibilitatea de la 99,9% la 100% în lunile de după.

Restul stivei

- Sute de microservicii, migrare de la monolit; Docker, **AWS ECS / Elastic Beanstalk**.
- Limbaje: Python 3 istoric, **Kotlin** pentru lucruri noi, Scala pentru motor.
- Date: DynamoDB, S3, MySQL/Postgres, plus EMR/Spark pentru batch.

- ML: **peste 300 de milioane de predicții pe zi**, PyTorch pe instanțe GPU.

Ce înseamnă pentru noi

Nimic din stivă nu ne e aplicabil și e important să spun asta explicit: voi aveți Next.js pe Vercel, Supabase, ~5 useri reali. Microserviciile, ECS și Spark ar fi sinucidere pentru o echipă de doi oameni. Duolingo a ajuns acolo *forțat*, după ani.

Ce e aplicabil e principiul, nu implementarea:

1. **„Elimină resursele partajate”** — la scara voastră se traduce în: conținutul (capitole/lecții/întrebări) se schimbă rar și se citește des. E candidatul perfect pentru cache, exact cum ei l-au mutat pe S3. La voi ar fi `use cache` din Next.js 16 sau ISR. Momentan fiecare `GET /api/chapters` lovește Supabase — vizibil în log-uri, ~200ms per cerere.
2. **Tiparea statică prinde bug-uri înainte de deploy** — voi aveți deja TypeScript strict + `tsc --noEmit` în CI. Aceeași lecție, învățată din start.
3. **Idempotența** la operațiuni declanșate extern — deja aplicată la webhook-ul Stripe (`processed_events`).

5b. Ce nu se vede — infrastructura din spate

Partea cea mai interesantă, pentru că nu apare în nicio recenzie de produs. Toate sunt din blogul lor de inginerie sau din prezentări tehnice.

Trasarea cererilor (request tracing) — dec. 2024

După trecerea la microservicii au pierdut vizibilitatea: într-un serviciu izolat loghezi ușor, dar într-un sistem distribuit nu mai știi cine pe cine cheamă.

- Au început cu **Jaeger** și trasare **eșantionată** (nu toate cererile).
- Problema eșantionării: bug-urile rare și „userul X are o problemă” rămân imposibil de reprodus, pentru că exact acea urmă lipsește.
- **Soluția lor, ingenioasă și ieftină:** în loc să construiască infrastructură nouă, au folosit ce aveau deja — antetul `X-Amzn-Trace-Id` pus de load balancerul AWS, propagat prin microservicii cu un câmp `Caller` adăugat de ei. Din log-urile existente, un **intern** a reconstruit stiva de apeluri cu AWS Athena + Python. Rezultat: **trasare pentru 100% din cereri**, procesată **la cerere**, nu continuu (procesarea continuă ar fi fost prea scumpă).
- **Ce au găsit imediat:** o metodă internă foarte des folosită interoga DynamoDB de mai multe ori pentru practic aceleași date, fără cache. Tiparul fusese rezonabil când a fost scris, dar se degradase în ani. Adăugarea unui cache: **–10% latență pe tot produsul** și reducere substanțială de cost.

Lecția, valabilă la orice scară: cea mai scumpă ineficiență nu e cea pe care o vezi, ci cea care a devenit normală în timp. Și n-au avut nevoie de infrastructură nouă ca s-o găsească — au folosit datele pe care le aveau deja.

DuoFactory — generarea conținutului cu LLM-uri

Un strat de orchestrare construit **peste Apache Airflow**, pentru toate fluxurile GenAI din companie. Exemplul documentat, fluxul „Duoradio”:

1. **Generare** — mai multe variante de scenariu, pornind de la datele de curriculum
2. **Evaluare cu LLM** — un al doilea model notează calitatea variantelor
3. **Selecție** — se alege cea mai bună
4. **Generare exerciții** — aceeași buclă generează-și-evaluează
5. **Publicare** — text-to-speech, sincronizare buze, încărcare pe S3

Rezultatele raportate: producția unui episod a scăzut de la **~o lună de muncă manuală la ~6 ore** (LLM + revizie umană), **–99% cost per episod, 243 de cursuri și peste 70.000 de episoade**, iar ascultătorii zilnici au crescut de 10×, de la 500.000 la 5 milioane în șase luni.

Oamenii non-tehnici administrează fluxurile prin editoare de prompturi, intrări din Google Sheets și un constructor de fluxuri fără cod.

Ingineria datelor (Data Refinery)

Tratează datele ca pe cod, cu practici pe care le-ați recunoaște din CI-ul vostru:

- **linting pe SQL** — convenții de denumire și tipuri impuse automat (ex. semnalează o coloană `event_timestamp` stocată ca `DATE` în loc de `TIMESTAMP`);
- **„data diffs” în pull request** — vezi cum se schimbă datele reale față de producție, înainte să dai merge;
- **deploy blue-green pentru date** — modificările se scriu în tabele separate, se umplu complet, apoi se clonează simultan în producție, ca să nu existe momente în care datele sunt parțiale sau înșelătoare;
- alerte pe Slack către dezvoltatori **și** către consumatorii datelor.

Agent AI care șterge feature flag-uri

Un detaliu mic dar revelator: au construit un agent care curăță automat feature flag-urile moarte din cod. Prototip în ~1 zi, versiune de producție în ~1 săptămână, majoritatea timpului consumată pe prompt engineering și extinderea la mai multe limbaje. Faptul că merită automatizat spune totul despre **volumul de experimente** pe care îl rulează: dacă ai 500 de experimente simultan, flag-urile moarte devin o problemă reală de întreținere.

Ce înseamnă pentru noi

- 1. Generarea de conținut cu LLM + revizie umană e cel mai direct transferabil lucru din toată cercetarea** — și atacă exact blocajul vostru real. Aveți „structura reală de capitole BAC” și „conținut real lecții” blocate de săptămâni pe disponibilitatea profesorului partener. Tiparul lor (generează mai multe variante → evaluează → alege → **omul revizuieste**, nu scrie de la zero) transformă rolul profesorului din *autor* în *revizor*. Un profesor care validează 20 de întrebări generate lucrează de câteva ori mai repede decât unul care le scrie. Nu vă trebuie Airflow — vă trebuie un script și o pagină de revizie în `/profesor`.
- 2. Trasarea la cerere, nu continuă.** Voi aveți deja `error_logs` și `/admin`. Ideea lor de a **nu** construi infrastructură nouă, ci de a extrage din ce există, e exact potrivită pentru doi oameni.
- 3. Cache-ul pe date citite des.** Ineficiența pe care au găsit-o (interogări repetate, necachate) o aveți deja, în mic: `GET /api/chapters` lovește Supabase la fiecare randare, ~200ms, pentru date care se schimbă o dată pe săptămână.
- 4. Un lucru pe care NU-l vom face:** blue-green pe date, linting SQL, agenți de curățat flag-uri. Sunt soluții la probleme pe care le ai la 500 de experimente și zeci de ingineri. Le notez ca să se vadă limita: **de la un punct încolo, ce citiți în blogul lor de inginerie e răspuns la probleme pe care e sănătos să nu le aveți.**

6. Cultura de experimentare

Aici e partea pe care aproape nimeni n-o copiază, deși e cea mai importantă.

Regulile lor (de la Gina Gotthilf, fost VP Growth — „*testăm A/B aproape tot și lăsăm metricile să ia majoritatea deciziilor*”):

- 1. Testează orice** — „chiar și un punct trimis userilor produce date valoroase”.
- 2. Prioritizează după ROI:** câți useri sunt afectați × cât de mare e câștigul. Prag: 1% pentru produs matur, **20-30% pentru startup**.
- 3. Maxim 3 variante** per experiment (control + 2), altfel statistica se strică.
- 4. Iterează pe câștiguri** — un test reușit naște următorul test.
- 5. Respectă valorile brandului** — unele teste nu se rulează chiar dacă ar câștiga.

Scara: 500+ experimente simultan; *produsul nu e identic pentru doi useri*. Prag minim pentru semnificație statistică: **100.000 DAU**. Recunosc că **~75% dintre idei sunt slabe** — dar le notează pe toate și le revizuiesc trimestrial.

Growth model: un **model Markov** care împarte userii în stări (nou, curent, reactivat, resuscitat) și urmărește tranzițiile zilnice; suma dă DAU. Concluzia lor cea mai citată: creșterea **CURR** (rata de retenție a userilor curenți) cu 2% pe lună a avut **cel mai mare impact — 75% din creșterea DAU pe trei ani**.

Ce înseamnă pentru noi

A/B testingul vă e inutil deocamdată și e cinstit s-o spunem: cu ~5 useri, orice diferență e zgomot. Le trebuie 100.000 DAU pentru semnificație; voi aveți nevoie de **alt instrument la scara voastră — interviul**. Cei 10-20 de elevi din Săpt. 11-12 vă vor spune în 30 de minute mai mult decât orice test statistic.

Ce e aplicabil imediat:

- **Ideea de „growth model”**, adaptată: nu DAU, ci **câți elevi au terminat capitolul X**. Adică exact ce dă tabelul `student_progress` — dar avem nevoie și de „câți au început și au abandonat”, ceea ce iar cere evenimente, nu doar starea finală.
- **Lista de idei cu revizuire periodică** — voi aveți deja `TASKS.md`. Funcționează.
- **Onestitatea că 75% dintre idei sunt slabe**. E o normă culturală sănătoasă.

7. Monetizare

- **Free**: complet funcțional, cu reclame. E deliberat: „mai mulți elevi = mai multe date = învățare mai bună”.
- **Super** (~85-95 \$/an în SUA): fără reclame, vieți nelimitate, Practice Hub.
- **Max** (~168 \$/an): tot ce e în Super + funcții AI (Video Call, Roleplay). „Explain My Answer” a devenit **gratuit pentru toți în 2026** — funcția AI care explică greșeala a fost mutată din tier-ul plătit în cel gratuit.
- Conversie: **~9,2%** din MAU. Concentrată puternic în userii zilnici, cu streak.
- De la lansarea Max (2023), venitul a crescut cu **peste 140%**.

Ce înseamnă pentru noi

Trei observații care contează pentru modelul vostru free/premium:

1. **Free-ul lor e generos, nu ciuntit**. Un elev poate învăța complet gratis. Plătește pentru confort, nu pentru acces. Modelul vostru actual (capitole premium blocate) e opusul. Nu spun că e greșit — piața românească de meditații e obișnuită cu plata — dar e o decizie strategică care merită luată conștient, nu implicit.
2. **Conversia vine din obișnuință, nu din blocaj**. Plătesc userii zilnici. Adică: valoarea se demonstrează întâi, se monetizează după.
3. **Au mutat funcția AI de explicare în free**. Semnal interesant: explicația greșelii e considerată **necesitate pedagogică**, nu lux. Pentru voi, unde „de ce am greșit” e miezul pregătirii de examen, merită gândit la fel — mai ales că aveți deja coloana `questions.explanation`, momentan neexploatăată în UI.

8. Chiar funcționează? (eficacitate)

Duolingo își publică studiile de eficacitate, cu teste standardizate independente:

- Terminarea conținutului **A2** ≈ **patru semestre universitare** la citit și ascultat.
- Jumătate din **B1** ≈ **cinci semestre universitare**.
- Drumul liniar („path”) **produce rezultate mai bune** decât vechiul arbore.

Merită reținut **că fac asta deloc**: măsoară dacă produsul chiar învață oameni, nu doar dacă îi ține în aplicație. Puține edtech-uri o fac.

Ce înseamnă pentru noi

Voi aveți un avantaj pe care Duolingo nu-l are: **nota la BAC e o măsurătoare externă, obiectivă și publică**. Duolingo a trebuit să inventeze teste ca să-și demonstreze eficacitatea. Voi va trebui doar să întrebați elevii ce notă au luat.

Asta e, pe termen lung, **cel mai valoros lucru pe care îl poate avea platforma**: „elevii care au terminat capitolele pe platformă au luat în medie nota X”. Dar cere de pe acum să reținem datele care fac legătura posibilă.

9. Sintează — ce luăm, ce nu

Idee Duolingo	Verdict	De ce
Conținut generat cu LLM + revizie umană	Luăm, prioritate 1	atacă blocajul real: profesorul devine revizor, nu autor
Istoric per răspuns (baza pentru tot)	Luăm, prioritate 1	fără el nu există „greșelile mele”, statistici pe întrebare, repetiție
„Greșelile mele”	Luăm	cea mai utilă funcție pentru un elev de examen, cost mic
Dificultate reală per întrebare (% greșeli)	Luăm	GROUP BY , zero ML, valoare mare pentru profesor
Streak redefinit + „îngheț”	Luăm, cu grijă	consecvența contează; pedeapsa nu
Progres vizibil per capitol	Luăm	răspunde la „sunt pregătit?”
Explicația greșelii, gratuită	Luăm	avem deja explanation în DB, nefolosit
Drum liniar obligatoriu	Parțial	recomandat, nu blocat — elevul de BAC are motive legitime să sară
Repetiție spațiată (HLR)	Mai târziu	valoros, dar cere istoric + volum de date
Notificări ca declanșator	Mai târziu	întâi emailul de tichet, apoi restul
Ligi / clasamente publice	Nu	stres inutil la un public deja stresat
Hearts / vieți	Nu	pedepsește exact ce vrei să încurajezi
XP ca metrică centrală	Nu	avem o metrică onestă: scorul pe capitol
A/B testing	Nu acum	le trebuie 100k DAU; nouă ne trebuie 20 de interviuri
Microservicii / ECS / Spark	Nu	sinucidere pentru doi oameni

Cea mai importantă concluzie tehnică: modelul nostru de date reține **stări** (`student_progress` suprascris la fiecare încercare), iar Duolingo reține **evenimente**. Aproape tot ce e valoros la ei — personalizare, repetiție, statistici, măsurarea eficacității — derivă din faptul că păstrează fiecare interacțiune. E o decizie de arhitectură pe care e mult mai ieftin s-o luăm acum, cu 5 useri, decât peste un an.

10. Întrebări deschise pentru discuție

1. **Free vs. premium:** copiem modelul „free generos, plătești pentru confort”, sau rămânem pe „capitole premium blocate”? Are consecințe asupra a tot ce urmează.
2. **Drum liniar sau acces liber** la capitole?
3. **Streak:** îl vrem? Cum îl definim ca să nu devină pedeapsă?
4. **Trecem pe model de evenimente** (`answer_events`) acum, cât e ieftin?

5. **Explicațiile la greșeli:** le arătăm imediat după test, sau doar la cerere?

6. **Măsurarea eficacității:** ne asumăm de pe acum să colectăm notele reale de BAC?

Surse

Duolingo — oficial

- [Rewriting Duolingo's engine in Scala](#) — arhitectura Session Generator, 750ms → 14ms
- [Duolingo Research](#) — lista de lucrări științifice
- [A Trainable Spaced Repetition Model for Language Learning](#) (Settles & Meeder, ACL 2016) — HLR
- [halflife-regression](#) pe GitHub — cod + date
- [Guide to Duolingo Practice Hub](#)
- [Improving the Duolingo experience with request tracing](#) — trasare 100% din cereri, -10% latență
- [How We Built an AI Agent to Remove Feature Flags](#)
- [Dataset development](#) — linting SQL, data diffs, blue-green pe date
- [Airflow in Action: Orchestrating AI At Duolingo](#) — DuoFactory, generare de conținut cu LLM, -99% cost/episod
- [Duolingo Path Meets Expectations for Proficiency Outcomes](#) — eficacitate
- [Duolingo Efficacy Studies](#)
- [Company Strategy Overview](#)
- [Q4/FY2025 shareholder letter](#) și Q1 2026

Analize și interviuri

- [The Tenets of A/B Testing from Duolingo's Master Growth Hacker](#) — First Round Review
- [How We Created a High-Scale Notification System at Duolingo](#) — InfoQ
- [Scala at Duolingo](#) — CoRecursive
- [The Duolingo Learning Path](#) — structura pe secțiuni/unități/niveluri
- [Duolingo Practice Hub](#)
- [Duolingo AI Personalization: How Birdbrain Works](#)
- [Duolingo Gamification Strategy: A Full Case Study](#)
- [Duolingo's Customer Retention Strategy](#)
- [Breaking Down DAU: Duolingo's Growth Model](#)

Notă despre calitatea surselor: cifrele de business și cele din blogul de inginerie vin din surse primare (rapoarte către investitori, blog oficial, lucrări peer-reviewed). Cifrele de gamificare (ex. „-21% churn de la Streak Freeze”) circulă prin bloguri de marketing și **nu au sursă primară verificabilă** — le-am păstrat pentru că indică ordinul de mărime, dar nu le-aș folosi ca argument decisiv.